# SEMANA 5 — AUDITORÍA COMPLETA + SEGURIDAD ANTI "ERRORES TÍPICOS"

## ÍNDICE

**Tarea 1 — Crear tabla audit_logs (auditoría).....2**
**Tarea 2 — Registrar auditoría en acciones clave..... 3**
**Tarea 3 — Pantalla "Auditoría" (solo Admin).....6**
**Tarea 4 — Protección CSRF + validaciones estrictas en formularios..... 10**
**Tarea 5 — Limitación de intentos de login (anti fuerza bruta simple)..... 14**
**Conclusión..... 17**

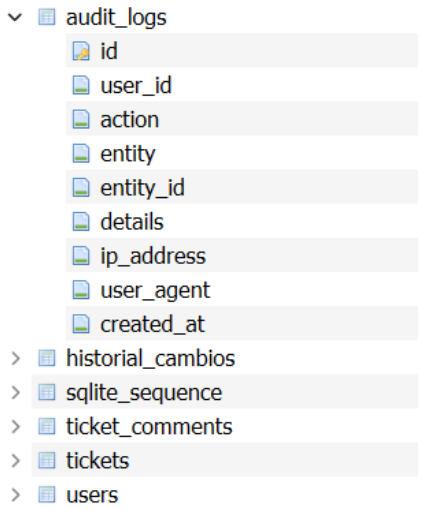
# Tarea 1 — Crear tabla audit_logs (auditoría)

El objetivo de esta primera tarea de la Semana 5 es establecer una estructura de base de datos que permita registrar de forma centralizada todas las acciones importantes que realizan los usuarios dentro de Secure Desk. Esta tabla, llamada **audit_logs**, será el fundamento de la auditoría del sistema.

La tabla se ha creado mediante la siguiente sentencia SQL, compatible con SQLite (el motor que utiliza Secure Desk):

```php
// ==================================================
// TABLA DE AUDITORIAS
// ==================================================
$db->exec("
        CREATE TABLE IF NOT EXISTS audit_logs (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        user_id INTEGER,
        action VARCHAR(50) NOT NULL,
        entity VARCHAR(50) NOT NULL,
        entity_id INTEGER,
        details TEXT NOT NULL,
        ip_address VARCHAR(45),
        user_agent TEXT,
        created_at DATETIME DEFAULT CURRENT_TIMESTAMP
    )
");
```

Esta sentencia se ha añadido al archivo **db/db_init.php**, junto con el resto de tablas del sistema, para mantener un único punto de inicialización de la base de datos. En mi caso para comprobar que como siempre se ha creado desde la herramienta **DB Browser for SQLite** he visualizado que se ha creado correctamente.



# Tarea 2 — Registrar auditoría en acciones clave

Implementar el registro automático de logs en las acciones más relevantes de Secure Desk: creación de tickets, edición de tickets (cambios en estado, prioridad o asignado), adición de comentarios, inicio de sesión (éxito y fallo) y cierre de sesión. Con esto se busca tener una trazabilidad completa de quién hizo qué, cuándo y desde dónde, cumpliendo con los requisitos de auditoría del sistema.

Antes de modificar los controladores, nos aseguramos de que existe una función centralizada para insertar registros en **audit_logs**. Esta función recibe los parámetros necesarios y **automáticamente captura la IP** y el **user agent** del cliente.

```php
<?php
// app/audit.php
require_once __DIR__ . '/config.php';

function registrar_auditoria($db, $userId, $action, $entity, $entityId, $details) {
    $ip = $_SERVER['REMOTE_ADDR'] ?? '0.0.0.0';
    $userAgent = $_SERVER['HTTP_USER_AGENT'] ?? '';

    $stmt = $db->prepare("
        INSERT INTO audit_logs (user_id, action, entity, entity_id, details, ip_address, user_agent)
        VALUES (:user_id, :action, :entity, :entity_id, :details, :ip, :ua)
    ");
    $stmt->execute([
        ':user_id' => $userId,
        ':action' => $action,
        ':entity' => $entity,
        ':entity_id' => $entityId,
        ':details' => $details,
        ':ip' => $ip,
        ':ua' => $userAgent
    ]);
}
```

Aceptamos **$user_id = null** para acciones de usuarios no autenticados (login fallido).

También se añadieron a cada uno de los campos que tendrá en cuenta la auditoría un controlador, con los cuales darán el mensaje de cambios en tickets, usuarios logueados,... Para que todos funcionen, se cargó en todos los documentos

En **“public\ticket_create.php”** se añadió la llamada a registrar_auditoria después de insertar el ticket y obtener su ID. También se incluyó validación de prioridad y manejo de excepciones.

```php
// Registrar auditoría de creación
$details = "Creó el ticket #$ticketId: " . substr($title, 0, 50) . (strlen($title) > 50 ? '...' : '');
registrar_auditoria($db, $_SESSION['user_id'], 'TICKET_CREATE', 'ticket', $ticketId, $details);
```

En **“public/ticket_detail.php”** se añadieron dos controladores:
**Edición**: dentro de la transacción, después de actualizar el ticket y antes del commit, se registra la acción.

```
// 3. Registrar auditoría de la edición
$details = "Actualizó el ticket #$id (campos: estado/prioridad/asignado)";
registrar_auditoria($db, $_SESSION['user_id'], 'TICKET_UPDATE', 'ticket', $id, $details);
```

**Comentario**: se añadió el bloque que procesaba **add_comment** (que faltaba) y se registra.

```
// Registrar auditoría del comentario
$details = "Añadió un comentario al ticket #$id";
registrar_auditoria($db, $_SESSION['user_id'], 'COMMENT_ADD', 'ticket', $id, $details);
```

Para las entradas, de nuevo se hicieron dos posibilidades distintas, una en la que el login es exitoso y otro fallido.
**Login exitoso**: después de verificar credenciales y antes de redirigir.

```
// Registrar auditoría de login exitoso
$details = "Inicio de sesión exitoso para usuario: " . $user['username'];
registrar_auditoria($db, $user['id'], 'LOGIN_SUCCESS', 'user', $user['id'], $details);
```

**Login fallido**: cuando las credenciales no coinciden, se registra con **user_id = null**.

```
// Login fallido: registrar auditoría (user_id = null)
$details = "Intento fallido de inicio de sesión para usuario: " . $username;
registrar_auditoria($db, null, 'LOGIN_FAILED', 'user', null, $details);
```

Y por último, cuando se va a cerrar sesion en **public/logout.php**, antes de destruir la sesión, si existe un usuario autenticado, se registra la acción:

```
// Registrar logout si hay un usuario autenticado
if (isset($_SESSION['user_id'])) {
    $userId = $_SESSION['user_id'];
    $username = $_SESSION['username'] ?? 'Desconocido';
    $details = "Cierre de sesión del usuario: $username";

    // Llamar a la función de auditoría (asume que está definida en audit.php)
    // Parámetros: ($db, $user_id, $action, $entity, $entity_id, $details)
    registrar_auditoria($db, $userId, 'LOGOUT', 'user', $userId, $details);
}
```

Para comprobar que los logs se están generando correctamente, se realizaron algunas acciones manuales y se consultó la tabla **audit_logs**, para comprobar que efectivamente se quedan registrados.

Tabla: audit_logs

| | id | user_id | action | entity | entity_id | details | ip_address | user_agent | cre |
|---|---|---|---|---|---|---|---|---|---|
| | Fil... | Filtro | Filtro | Filtro | Filtro | Filtro | Filtro | Filtro | Filtro |
| 1 | 1 | 1 | TICKET_UPDATE | ticket | 1 | Actualizó el ticket #1 (campos: … | ::1 | Mozilla/5.0 (Windows NT 10.0; Win64;… | 2026-02- |
| 2 | 2 | 1 | LOGOUT | user | 1 | Cierre de sesión del usuario: admin | ::1 | Mozilla/5.0 (Windows NT 10.0; Win64;… | 2026-02- |
| 3 | 3 | NULL | LOGIN_FAILED | user | NULL | Intento fallido de inicio de sesión … | ::1 | Mozilla/5.0 (Windows NT 10.0; Win64;… | 2026-02- |
| 4 | 4 | 1 | LOGIN_SUCCESS | user | 1 | Inicio de sesión exitoso para … | ::1 | Mozilla/5.0 (Windows NT 10.0; Win64;… | 2026-02- |

# Tarea 3 — Pantalla “Auditoría” (solo Admin)

El objetivo de esta tarea es crear un panel de consulta de logs de auditoría exclusivo para administradores, que permita visualizar toda la actividad registrada en el sistema y filtrar por usuario y por acción. Este panel debe ser accesible únicamente por usuarios con rol **admin** y proporcionar una interfaz clara, moderna y funcional.

Como muchas veces, hemos separado la **capa lógica** que se encuentra en **public/**, de la **capa visual** en **views/** a continuación explicaremos que hemos hecho en ambos documentos.

Empezando por **public/audit.php** el archivo sigue el patrón que ya utilizas en otras vistas, como en por ejemplo el apartado de usuarios, tenemos un controlador que solo deja acceder a los usuarios admins dentro de este apartado, en caso de no serlo se les redirige automáticamente a el **login.php**.

```php
// Solo para admins, redirige en caso de no serlo
if (!isset($_SESSION['user_id']) || $_SESSION['role'] !== 'admin') {
    header('Location: ../public/login.php');
    exit;
}
```

Tras esto importamos el archivo **app/audit.php** y **app/config.php**, y obtenemos la lista de usuarios y de acciones.

```php
// Obtener lista de usuarios para el filtro (id => username)
$stmt = $db->query("SELECT id, username FROM users ORDER BY username");
$users = $stmt->fetchAll(PDO::FETCH_KEY_PAIR);

// Obtener lista de acciones distintas registradas
$stmt = $db->query("SELECT DISTINCT action FROM audit_logs ORDER BY action");
$actions = $stmt->fetchAll(PDO::FETCH_COLUMN);
```

Para que funcione, se aplican los filtros si existen en **$_GET**, usando marcadores de parámetros para evitar inyecciones SQL.

```php
// Construir consulta con filtros
$where = [];
$params = [];

if (!empty($_GET['user_id'])) {
    $where[] = "audit_logs.user_id = :user_id";
    $params[':user_id'] = $_GET['user_id'];
}
if (!empty($_GET['action'])) {
    $where[] = "audit_logs.action = :action";
    $params[':action'] = $_GET['action'];
}
```

Y se ejecuta una consulta dependiendo de los parámetros seleccionados, la consulta final incluye un **LEFT JOIN** con **users** para obtener el nombre del usuario (puede ser **NULL** si el usuario fue eliminado o era un intento de login fallido).

Tras la consulta lógicamente, debemos ejecutarla para obtener los parámetros:

```php
$stmt = $db->prepare($sql);
$stmt->execute($params);
$logs = $stmt->fetchAll(PDO::FETCH_ASSOC);
```

También se le ha añadido el menú, y en el menú se ha añadido un apartado que solo pueden ver los usuarios admins que redirige a las pestaña de auditoría y por último, tras ejecutar la consulta, carga la vista de auditoría **views/audit.php**.

Tras esto, pasamos a la parte de la vista, le hemos dado un estilo moderno (ayudandome de **deepseek**, para que mentir, me lo ha hecho bastante bien y no pierdo apenas tiempo) hemos creado un formulario y una tabla donde se muestran los resultados que explicaré a continuación.

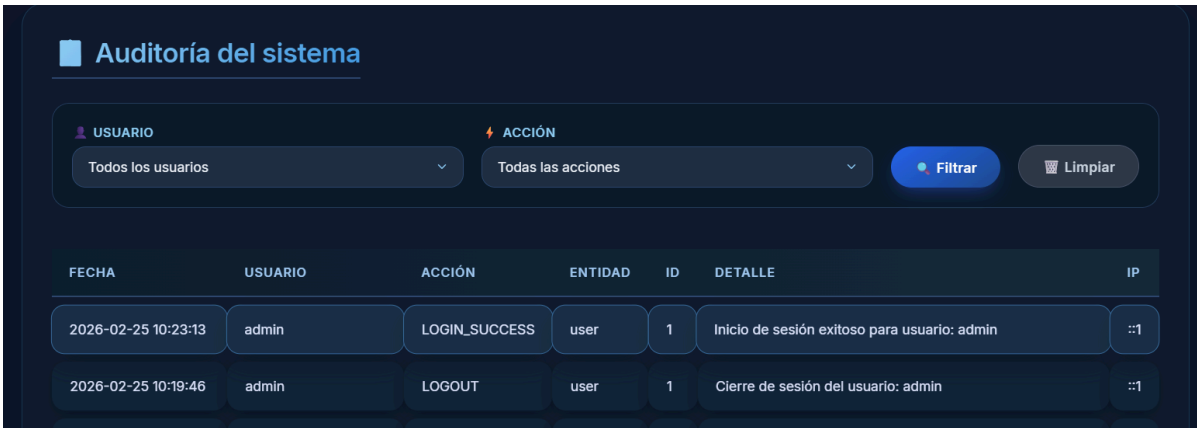
Comenzando por hacer el formulario donde seleccionamos el usuario y acción que queremos filtrar, un botón de filtrar que ejecuta la consulta y uno que es limpiar que recarga la página haciendo que los filtros vuelvan a estar en “**todo seleccionado**”.

```
<!-- Formulario de filtros -->
<form method="GET" class="filters">
    <div class="filter-group">
        <label for="user_id">👤 Usuario</label>
        <select name="user_id" id="user_id">
            <option value="">Todos los usuarios</option>
            <?php foreach ($users as $id => $username): ?>
                <option value="<?= $id ?>" <?= (isset($_GET['user_id']) && $_GET['user_id'] == $id) ? 'selected' : '' ?>>
                    <?= htmlspecialchars($username) ?>
                </option>
            <?php endforeach; ?>
        </select>
    </div>

    <div class="filter-group">
        <label for="action">⚡ Acción</label>
        <select name="action" id="action">
            <option value="">Todas las acciones</option>
            <?php foreach ($actions as $act): ?>
                <option value="<?= htmlspecialchars($act) ?>" <?= (isset($_GET['action']) && $_GET['action'] === $act) ? 'selected' : '' ?>>
                    <?= htmlspecialchars($act) ?>
                </option>
            <?php endforeach; ?>
        </select>
    </div>

    <button type="submit" class="btn-filter">🔍 Filtrar</button>
    <a href="audit.php" class="btn-clear">🗑 Limpiar</a>
</form>
```

La tabla de resultados se construye dinámicamente con **foreach ($logs as $log)**. Cada columna muestra un campo del registro, aplicando **htmlspecialchars()** para evitar XSS.

- **Fecha**: $log['created_at']
- **Usuario**: Si $log['username'] es null (por ejemplo, en intentos de login fallidos), se muestra "Sistema/Desconocido".
- **Acción**: Se añade un atributo data-action para posibles estilos personalizados.
- **Entidad**: $log['entity']
- **ID de entidad**: Si es null, se muestra un guión (-).
- **Detalle**: Texto descriptivo.
- **IP**: Si es null, se muestra un guión.

```php
<!-- Resultados -->
<?php if (empty($logs)): ?>
    <div class="no-results">
        ✨ No hay registros de auditoría que coincidan con los filtros.
    </div>
<?php else: ?>
    <table class="audit-table">
        <thead>
            <tr>
                <th>Fecha</th>
                <th>Usuario</th>
                <th>Acción</th>
                <th>Entidad</th>
                <th>ID</th>
                <th>Detalle</th>
                <th>IP</th>
            </tr>
        </thead>
        <tbody>
            <?php foreach ($logs as $log): ?>
                <tr>
                    <td><?= htmlspecialchars($log['created_at']) ?></td>
                    <td><?= htmlspecialchars($log['username'] ?? 'Sistema/Desconocido') ?></td>
                    <td data-action="<?= htmlspecialchars($log['action']) ?>"><?= htmlspecialchars($log['action']) ?></td>
                    <td><?= htmlspecialchars($log['entity']) ?></td>
                    <td><?= htmlspecialchars($log['entity_id'] ?? '-') ?></td>
                    <td><?= htmlspecialchars($log['details']) ?></td>
                    <td><?= htmlspecialchars($log['ip_address'] ?? '-') ?></td>
                </tr>
            <?php endforeach; ?>
        </tbody>
    </table>
<?php endif; ?>
```



# Tarea 4 — Protección CSRF + validaciones estrictas en formularios

Reforzar la seguridad e integridad de los datos en Secure Desk mediante la implementación de tokens CSRF en todos los formularios críticos y la validación rigurosa de campos obligatorios en el servidor, complementada con la sanitización de la salida para prevenir ataques XSS.

Para comenzar he tenido que investigar que es un CSRF, ya que no me sonaba y he tenido que ver en qué consiste, cómo se hace,... A la conclusión que he llegado es que el **Cross-Site Request Forgery** (CSRF) es un tipo de ataque que fuerza a un usuario autenticado a ejecutar acciones no deseadas en una aplicación web. Por ejemplo, un atacante podría engañar a un administrador para que, sin darse cuenta, cree un ticket o modifique uno existente.

Tras esto, me he puesto manos a la obra sobre como puedo evitar estos ataques, la creación de los tokens en los formularios,... Y se ha creado un sistema de tokens CSRF que funciona de la siguiente manera:

- **Generación**: En cada controlador que muestra un formulario, se genera un token único mediante **bin2hex(random_bytes(32))** y se almacena en la sesión.
- **Inclusión**: El token se inserta como un campo oculto en el formulario.
- **Verificación**: Al recibir la petición **POST**, se compara el token recibido con el almacenado en sesión. Si no coinciden, la petición es rechazada.

Todo este mecanismo se ha centralizado en el nuevo archivo **app/security.php**, que contiene las funciones:

- **generar_csrf_token()** – Crea o recupera el token de sesión.
- **verificar_csrf_token($token)** – Comprueba la validez del token.
- **limpiar_csrf_token()** – Elimina el token de la sesión (opcional).

```php
/**
 * Genera un token CSRF y lo guarda en la sesión.
 * @return string Token generado.
 */
function generar_csrf_token() {
    if (empty($_SESSION['csrf_token'])) {
        $_SESSION['csrf_token'] = bin2hex(random_bytes(32));
    }
    return $_SESSION['csrf_token'];
}

/**
 * Verifica que el token proporcionado coincida con el de la sesión.
 * @param string $token Token recibido del formulario.
 * @return bool True si es válido, false en caso contrario.
 */
function verificar_csrf_token($token) {
    return isset($_SESSION['csrf_token']) && hash_equals($_SESSION['csrf_token'], $token);
}

/**
 * Limpia el token CSRF de la sesión (opcional, después de usarlo).
 */
function limpiar_csrf_token() {
    unset($_SESSION['csrf_token']);
}
```

Tras esto, se añadieron diferentes controladores en **public/login.php**, **views/login.php**, **public/ticket_create.php**, **views/tickets/create.php**, **public/ticket_detail.php** y **views/tickets/detail.php**.

Comenzando por **public/login.php**:

- Se añadió **require_once __DIR__ . '/../app/security.php'**;
- Se genera el token antes de cargar la vista: **$csrf_token = generar_csrf_token();**
- Se mantienen las validaciones de campos vacíos y la auditoría.
- En el bloque **POST**, lo primero es verificar el token:

```php
// Verificar token CSRF
if (!isset($_POST['csrf_token']) || !verificar_csrf_token($_POST['csrf_token'])) {
    die("Petición no válida (token CSRF).");
}
```

En la vista del login (**views/login.php**) también se añadieron cambios:

- Se mejoró el diseño con estilos modernos (tarjeta centrada, gradientes, efectos hover).
- Se añadió el campo oculto donde se rellena con el token generado:

```php
<!-- Token CSRF: generado en el controlador y pasado a la vista.
     Este token debe coincidir con el almacenado en sesión para que la petición sea válida. -->
<input type="hidden" name="csrf_token" value="<?= htmlspecialchars($csrf_token ?? '') ?>">
```

También se añadieron controladores en **public/ticket_create.php**:

- Se incluyó **security.php**.
- Generación del token al inicio.
- Verificación del token en el **POST** antes de cualquier otra validación.
- Se pasó **$csrf_token** a la vista.
- Validaciones estrictas en servidor:

```php
// --- Validaciones estrictas en servidor ---
if (empty($title)) {
    $error = 'El título es obligatorio.';
} elseif (empty($description)) {
    $error = 'La descripción no puede estar vacía.';
} elseif (empty($priority)) {
    $error = 'Debes seleccionar una prioridad.';
} elseif (!in_array($priority, TICKET_PRIORITY)) {
    $error = 'Prioridad no válida.';
} else {
```

En la vista de tickets, se hicieron cosas similares al login, así que los cambios en **views/tickets/create.php** fueron los siguientes:

- Campo oculto con el token.
- Atributos **required** en los campos obligatorios (título, descripción, prioridad) para validación en cliente.
- Mejora del estilo para que quede más homogéneo a todo lo demás de la web.

Lo siguiente en modificar y último, fue la edición de los tickets, comenzando por la parte lógica **public/ticket_detail.php**:

- Se añadió **require_once __DIR__ . '/../app/security.php'**;
- Generación del token al inicio (se pasa a la vista).
- Validaciones de campos obligatorios:
    - **En edición**: se comprueba que **$description** no esté vacía (además de estado y prioridad contra constantes).
    - **En comentario**: **if ($comment === '') { $error = "El comentario no puede estar vacío."; }**
- Verificación del token en ambos bloques **POST** (**edit_ticket** y **add_comment**):

```php
// Verificar token CSRF (lo primero)
if (!isset($_POST['csrf_token']) || !verificar_csrf_token($_POST['csrf_token'])) {
    die("Error de validación CSRF. Intenta de nuevo.");
}
```

- En la vista de la edición de tickets (**views/tickets/detail.php**) se hicieron las siguientes modificaciones:
  Se añadió **required** en los campos de estado, prioridad, descripción y comentario.
- Se añadió el token CSRF en los dos formularios:

```
<!-- Token CSRF (generado en el controlador) -->
<input type="hidden" name="csrf_token" value="<?= htmlspecialchars($csrf_token) ?>">
```

Como se ha ido comentando a lo largo de las ediciones, los campos como títulos, usuarios, contraseñas,... Se les ha añadido el atributo **required**, lo que hace que si o si, se les deba de escribir algo para que se guarde (las validaciones)

También para la **sanitización de salida (XSS)** en todas las vistas se ha utilizado sistemáticamente **htmlspecialchars()** al mostrar cualquier dato proveniente del usuario o de la base de datos. Ejemplos:

- <?= htmlspecialchars($ticket['title']) ?>
- <?= nl2br(htmlspecialchars($c['comment'])) ?>
- <?= htmlspecialchars($item['data']['username']) ?>

Esto convierte caracteres especiales (<, >, &, etc.) en entidades HTML, impidiendo que se inyecte código malicioso.
Además, en campos que contienen texto largo (descripción, comentarios) se utiliza **nl2br()** después de **htmlspecialchars()** para respetar los saltos de línea sin riesgo.

# Tarea 5 — Limitación de intentos de login (anti fuerza bruta simple)

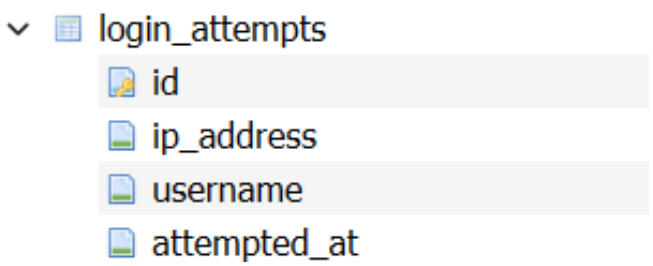
El objetivo de esta tarea es implementar un mecanismo que limite los intentos fallidos de inicio de sesión para prevenir ataques de fuerza bruta. Tras un número determinado de fallos (5 intentos en 5 minutos), se bloquea temporalmente el acceso desde esa **IP**, registrando el evento en auditoría.

Para comenzar, añadimos una nueva tabla **login_attempts** con la siguiente estructura:

- **id**: Identificador único del intento.
- **ip_address**: IP desde la que se intenta acceder (se normaliza a **127.0.0.1** si es **::1**).
- **username**: Nombre de usuario intentado (opcional, para análisis).
- **attempted_at**: Marca de tiempo automática (se usa para limpiar intentos antiguos).

```php
// ==================================================
// TABLA LOGIN_ATTEMPTS: control de intentos fallidos de login (anti fuerza bruta)
// ==================================================
$db->exec("
    CREATE TABLE IF NOT EXISTS login_attempts (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        ip_address VARCHAR(45) NOT NULL,
        username VARCHAR(100),
        attempted_at DATETIME DEFAULT CURRENT_TIMESTAMP
    )
");
// Índice para acelerar consultas por IP y fecha (opcional pero recomendado)
$db->exec("CREATE INDEX IF NOT EXISTS idx_login_attempts_ip ON login_attempts(ip_address)");
$db->exec("CREATE INDEX IF NOT EXISTS idx_login_attempts_attempted_at ON login_attempts(attempted_at)");
```

Como siempre, se hizo a través de **db\db_init.php**, metimos la ruta en el **localhost** y se genera la tabla, desde **DB Browser** podemos ver que se ha creado perfectamente.



Tras esto se empieza a trabajar en la lógica que tendrá los 5 intentos en **public/login.php**, comenzando por editar el código para que **obtenga la dirección IP y la normalice a IPv6**.

```
$ip = $_SERVER['REMOTE_ADDR'] ?? '0.0.0.0';
if ($ip === '::1') $ip = '127.0.0.1'; // Normalizar IPv6 local
```

Tras esto, se hace una **limpieza de los intentos** (aquellas IP que llevan más de 5 minutos sin intentarlo de nuevo) ya que en caso de hacerse más tarde, no dejaría hacer el intento, ya que seguiría registrado en la tabla de intentos fallidos.

```
// Limpiar intentos antiguos usando la hora de SQLite (más fiable, no depende de la zona horaria)
$stmt = $db->prepare("DELETE FROM login_attempts WHERE ip_address = :ip AND attempted_at < datetime('now', '-5 minutes')");
$stmt->execute([':ip' => $ip]);
```

Se usa **datetime('now', '-5 minutes')** de SQLite para evitar problemas de zona horaria con PHP.

Tras la limpieza, se hace un conteo de los números de fallos existentes.

```
// Contar intentos recientes
$stmt = $db->prepare("SELECT COUNT(*) as total FROM login_attempts WHERE ip_address = :ip");
$stmt->execute([':ip' => $ip]);
$intentos_recientes = $stmt->fetch(PDO::FETCH_ASSOC)['total'];

$max_intentos = 5;
$bloqueado = $intentos_recientes >= $max_intentos;
```

Antes de validar credenciales, se comprueba si la IP está bloqueada:

```
} elseif ($bloqueado) {
    //Mensaje y bloqueo de POST por exceso de fallos
    $details = "Intento de login bloqueado por exceso de fallos (IP: $ip)";
    registrar_auditoria($db, null, 'LOGIN_BLOCKED', 'system', null, $details);
    $error = "Demasiados intentos fallidos. Espera 5 minutos.";
```

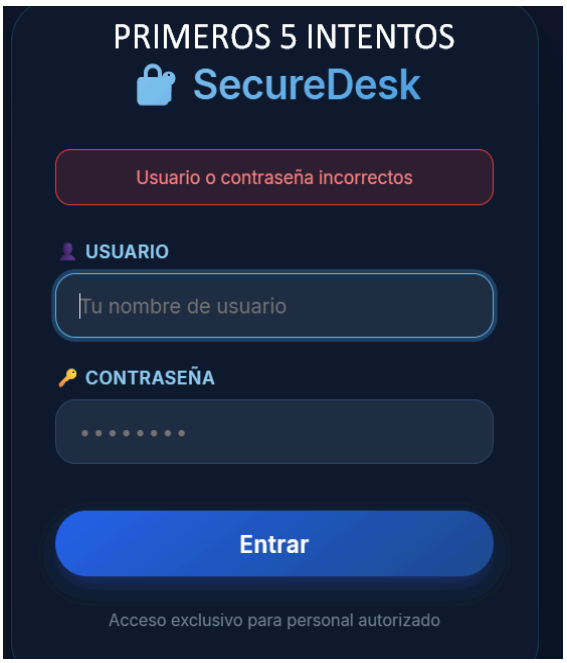
Cuando falla y las credenciales son incorrectas o el usuario no existe, se inserta un nuevo intento:

```php
} else {
    // Login fallido
    $stmt = $db->prepare("INSERT INTO login_attempts (ip_address, username) VALUES (:ip, :user)");
    $stmt->execute[':ip' => $ip, ':user' => $username]);

    $details = "Intento fallido de inicio de sesión para usuario: " . $username . " (IP: $ip)";
    registrar_auditoria($db, null, 'LOGIN_FAILED', 'user', null, $details);

    $error = 'Usuario o contraseña incorrectos';
}
```

Si el login es correcto, se borran todos los intentos de esa IP para no arrastrar bloqueos previos:

```php
    // Login exitoso
    $stmt = $db->prepare("DELETE FROM login_attempts WHERE ip_address = :ip");
    $stmt->execute[':ip' => $ip]);
```

Con esto, queda acabado y se realizan las pruebas (se han realizado a lo largo de que se ha ido haciendo, pero estas pruebas son para enseñar que efectivamente funciona).



Al esperar los 5 minutos se intenta de nuevo y se entra sin problema (con la contraseña correcta).

# Conclusión

A lo largo de esta semana hemos transformado Secure Desk en una aplicación mucho más robusta y profesional. Comenzamos sentando las bases de la auditoría con la creación de la tabla audit_logs, y acto seguido implementamos el registro automático de todas las acciones clave: inicios de sesión (exitosos y fallidos), cierres de sesión, creación y edición de tickets, y comentarios. Esto nos permite tener un historial detallado de quién hizo qué y desde dónde, algo fundamental.

También reforzamos la aplicación contra vulnerabilidades comunes. Añadimos protección CSRF en todos los formularios críticos, reforzamos las validaciones en servidor para evitar datos incorrectos o maliciosos, e implementamos un sistema de limitación de intentos de login que bloquea temporalmente IPs tras varios fallos, dificultando enormemente los ataques de fuerza bruta.

En algunas pestañas también se han mejorado las vistas, para que la web se vea mejor, más intuitiva y moderna.

Código completo en el GitHub:
https://github.com/elgancho06/Practicas-DAM